

# Modelos Generativos: Difusão Condicionada, VQ-VAE e Stable Diffusion

Machine Learning 2 2026.1

**Francisco Ganacim**, Daniel Yukimura

IMPA

May 27, 2026

# Roteiro da Aula

## 1. Síntese em Difusão Condicionada

- ▶ Com classificador (*Classifier Guidance*)
- ▶ Sem classificador (*Classifier-Free Guidance*)

## 2. VQ-VAE

- ▶ Representações discretas
- ▶ Quantização vetorial e *prior* autorregressivo

## 3. Stable Diffusion (*Latent Diffusion Models*)

- ▶ Autoencoder perceptual
- ▶ Difusão no espaço latente
- ▶ Condicionamento por *cross-attention*

---

# Difusão Condicionada

---

# Difusão Condicionada: Por quê?

Vimos que o modelo de difusão aprende

$$P(X|\theta) = \int P(X, Z_{1...T}|\theta) dZ_{1...T}$$

Porém, ao amostrar  $Z_T \sim \mathcal{N}(0, I)$ , não temos **controle** sobre o que é gerado.

- ▶ Queremos gerar imagens de uma **classe** específica.
- ▶ Queremos gerar imagens a partir de um **texto**.
- ▶ Queremos gerar imagens a partir de uma **máscara de segmentação**.

Em outras palavras, queremos aprender  $P(X|Y, \theta)$ , onde  $Y$  é a informação de condicionamento.

# Difusão Condicionada: Por quê?

Considere o problema de gerar **cachorros**, mas tendo o modelo aprendido sobre **todas** as imagens do ImageNet.

- ▶ Amostrar  $Z_T \sim \mathcal{N}(0, I)$  e percorrer o processo reverso gera uma imagem aleatória de qualquer classe.
- ▶ Se quisermos um cachorro: rejeitar amostras até obter um? **Caro.**
- ▶ Solução: modificar o processo reverso para favorecer a classe desejada.

Duas abordagens populares:

1. **Classifier Guidance**: usa um classificador externo para guiar o processo.
2. **Classifier-Free Guidance**: aprende o condicionamento dentro do próprio modelo.

# Classifier Guidance: Como?

**Ideia central:** usar o gradiente de um classificador  $P_\phi(Y|Z_t)$  para empurrar o processo reverso na direção da classe  $Y$ .

Pela regra de Bayes (em escala log):

$$\begin{aligned}\log P(Z_t|Y) &= \log P(Z_t) + \log P(Y|Z_t) - \log P(Y) \\ \nabla_{Z_t} \log P(Z_t|Y) &= \underbrace{\nabla_{Z_t} \log P(Z_t)}_{\text{modelo de difusão}} + \underbrace{\nabla_{Z_t} \log P(Y|Z_t)}_{\text{classificador}}\end{aligned}$$

O segundo termo é a derivada do log da probabilidade da classe  $Y$  dada a imagem ruidosa  $Z_t$ .

# Intuição: o Score é a Direção de Subida da Densidade

Antes de condicionar, lembremos **o que move** o processo reverso.

Seja como dinâmica de Langevin, seja como SDE reversa, cada passo empurra  $Z_t$  na direção do **score**:

$$\nabla_{Z_t} \log P(Z_t)$$

o gradiente do log da densidade **em relação a**  $Z_t$  — aponta para onde a amostra fica **mais plausível**.

O próprio preditor de ruído **é** esse score (a menos de escala):

$$\nabla_{Z_t} \log P(Z_t) \approx -\frac{1}{\sqrt{1-\alpha_t}} g_t(Z_t, \theta_t)$$

**Condicionar**  $\Rightarrow$  trocar o score  $\nabla_{Z_t} \log P(Z_t)$  pelo score **condicional**  $\nabla_{Z_t} \log P(Z_t|Y)$  no **mesmo** passo de amostragem.

Derivamos em  $Z_t$  porque é a variável que percorremos:  $Y$  e  $\theta$  são fixos. Como veremos, derivar em  $Z_t$  ainda **elimina** o termo intratável  $\log P(Y)$ .

# Classifier Guidance: Conexão com o Score

A parametrização do ruído define a marginal direta como uma **gaussiana**:

$$Z_t = \sqrt{\alpha_t}X + \sqrt{1 - \alpha_t}\epsilon \implies P(Z_t|X) = \mathcal{N}(Z_t; \sqrt{\alpha_t}X, (1 - \alpha_t)I)$$

O score de uma gaussiana é  $-(\text{ponto} - \text{média})/\text{variância}$ :

$$\nabla_{Z_t} \log P(Z_t|X) = -\frac{Z_t - \sqrt{\alpha_t}X}{1 - \alpha_t} = -\frac{\sqrt{1 - \alpha_t}\epsilon}{1 - \alpha_t} = -\frac{\epsilon}{\sqrt{1 - \alpha_t}}$$

O **denoiser** é treinado para estimar o ruído,  $g_t(Z_t, \theta_t) \approx \mathbb{E}[\epsilon|Z_t]$ .

Tomando a esperança (Tweedie), o score da **marginal** fica:

$$\nabla_{Z_t} \log P(Z_t) \approx -\frac{1}{\sqrt{1 - \alpha_t}} g_t(Z_t, \theta_t)$$





# Classifier Guidance: Desvantagens

- ▶ Requer **treino de um classificador adicional** sobre imagens ruidosas.
- ▶ Um classificador pré-treinado em imagens limpas **não serve**.
- ▶ Não é trivial estender para condicionamento contínuo (e.g. **texto**).
- ▶ O gradiente  $\nabla \log P_\phi(Y|Z_t)$  pode produzir *adversarial-like artifacts*.

**Motivação:** podemos atingir o mesmo efeito **sem um classificador separado**?

# Difusão Condicionada: Como $Y$ Entra na Rede?

O **backbone** é sempre a mesma **UNet** de difusão (*skips*, blocos residuais, *self-attention* nas baixas resoluções). Muda apenas **onde injetamos** a informação.

**Tempo  $t$**  (sempre): embedding sinusoidal  $\rightarrow$  MLP  $\rightarrow$  vetor **somado** em cada bloco residual (estilo FiLM / *adaptive group norm*).

**Condicionamento  $Y$** : três vias, conforme a modalidade:

- ▶ **Classe / vetor global**: embedding de  $Y$  **somado ao de  $t$** ;  $\emptyset$  = entrada extra aprendida. (caso de Ho & Salimans)
- ▶ **Texto / sequência**:  $\tau_\theta(y) \rightarrow$  *cross-attention*;  $\emptyset$  = embedding da string vazia.
- ▶ **Espacial** (máscara, *low-res*): **concatenado nos canais** de  $Z_t$ ;  $\emptyset$  = canais nulos.

Para o CFG: a **mesma rede, mesmos pesos** processa  $Y$  real ou  $\emptyset$ .

- [4] 2022 — Jonathan Ho et al. — *Classifier-Free Diffusion Guidance*  
[1] 2021 — Prafulla Dhariwal et al. — *Diffusion Models Beat GANs on Image Synthesis*  
[8] 2022 — Robin Rombach et al. — *High-Resolution Image Synthesis with Latent Diffusion Models*

# Classifier-Free Guidance: Como?

**Ideia:** treinar simultaneamente o modelo **condicional** e **incondicional** dentro da mesma rede.

Aplicando Bayes novamente:

$$\nabla_{Z_t} \log P(Y|Z_t) = \nabla_{Z_t} \log P(Z_t|Y) - \nabla_{Z_t} \log P(Z_t)$$

Substituindo na expressão de guidance, temos

$$\tilde{g}_t(Z_t, \theta_t, Y) = g_t(Z_t, \theta_t) - s\sqrt{1 - \alpha_t} [\nabla \log P(Z_t|Y) - \nabla \log P(Z_t)]$$

Que pode ser reescrito como uma combinação linear de **duas chamadas** do modelo de difusão:

$$\tilde{g}_t(Z_t, \theta_t, Y) = (1 + w) g_t(Z_t, \theta_t, Y) - w g_t(Z_t, \theta_t, \emptyset)$$

Onde  $\emptyset$  é o **token de condicionamento vazio** e  $w \geq 0$  é a escala de guidance.



# Classifier-Free Guidance: Treinamento

A rede recebe agora dois inputs: a imagem ruidosa  $Z_t$  e o condicionamento  $Y$ .

**Truque do treino:** com probabilidade  $p_{\text{uncond}}$  ( $\approx 10\text{--}20\%$ ), descartamos  $Y$  e treinamos com  $Y = \emptyset$ .

## Pseudocódigo de treino:

- ▶ Para cada exemplo  $(X^{(i)}, Y^{(i)})$  no batch:
  - ▶ Amostra  $t \sim \mathcal{U}(1, T)$
  - ▶ Amostra  $\epsilon \sim \mathcal{N}(0, I)$
  - ▶ Computa  $Z_t = \sqrt{\alpha_t} X^{(i)} + \sqrt{1 - \alpha_t} \epsilon$
  - ▶ Com prob.  $p_{\text{uncond}}$ :  $Y^{(i)} \leftarrow \emptyset$
  - ▶ Loss:  $\ell_i = \left\| g_t(Z_t, \theta_t, Y^{(i)}) - \epsilon \right\|^2$

A mesma rede aprende a prever ruído **condicional** e **incondicional**.

# Classifier-Free Guidance: Geração

- ▶ Amostra  $Z_T \sim \mathcal{N}(0, I)$
- ▶ Para  $t = T \dots, 1$ :
  - ▶ Avalia **duas vezes** o modelo:

$$g_t^c = g_t(Z_t, \theta_t, Y) \quad (\text{condicional})$$

$$g_t^u = g_t(Z_t, \theta_t, \emptyset) \quad (\text{incondicional})$$

- ▶ Combina:  $\tilde{g}_t = (1 + w) g_t^c - w g_t^u$
- ▶ Amostra  $\epsilon \sim \mathcal{N}(0, I)$  se  $t > 1$ , senão  $\epsilon = 0$
- ▶  $Z_{t-1} = \frac{1}{\sqrt{1-\beta_t}} Z_t - \frac{\sqrt{\beta_t}}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}} \tilde{g}_t + \sigma_t \epsilon$

**Custo:** dobra o número de *forward passes* em inferência.

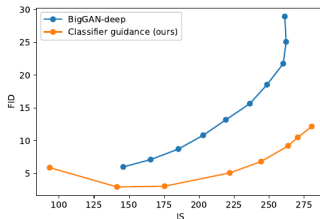
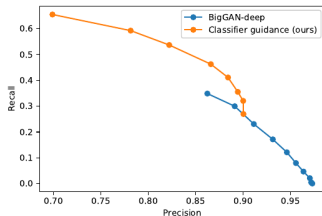


# Classifier Guidance: Efeito da Escala $s$



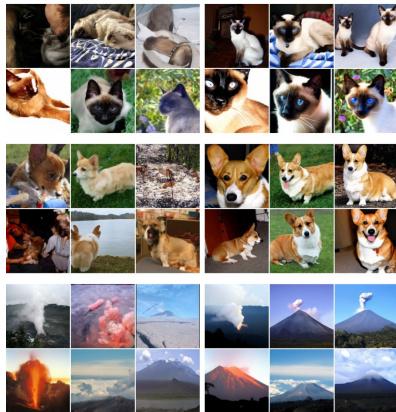
Esq.:  $s=1.0$  (FID=33.0, maior diversidade). Dir.:  $s=12.0$  (FID=12.0, maior fidelidade à classe). Classe: Pembroke Welsh Corgi.

# Classifier Guidance: Trade-off Precisão/Recall



Guidance aumenta precisão (fidelidade à classe) ao custo de recall (diversidade). Difusão com guidance supera BigGAN na fronteira de Pareto em ImageNet  $128 \times 128$ .

# Classifier-Free Guidance: Amostras



Esq.: sem guidance ( $w=0$ ). Dir.: com guidance ( $w=3.0$ ). ImageNet  $128 \times 128$ . Amostras guiadas são visivelmente mais consistentes com a classe, mas ligeiramente menos diversas.

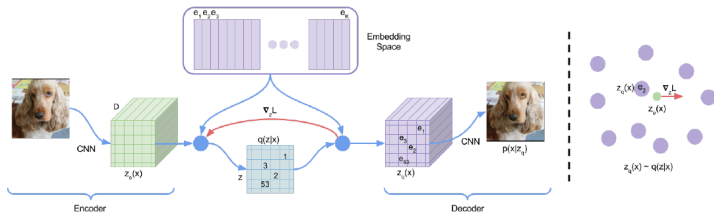
---

# VQ-VAE

---



# VQ-VAE: Arquitetura



Esq.: encoder CNN  $\rightarrow$  quantização ao vizinho mais próximo  $e_{k^*} \rightarrow$  decoder CNN. Dir.: gradiente flui pelo *straight-through* (seta vermelha) de volta ao encoder.

# VQ-VAE: Visão Geral

- ▶ **Codebook:**  $\mathcal{E} = \{e_1, e_2, \dots, e_K\}$ , com  $e_k \in \mathbb{R}^D$ .
- ▶ **Encoder:**  $z_e(X) = \text{enc}(X) \in \mathbb{R}^D$  (ou tensor  $h \times w \times D$  para imagens).
- ▶ **Quantização vetorial:** substituímos  $z_e(X)$  pelo **vizinho mais próximo** no codebook,

$$z_q(X) = e_{k^*}, \quad k^* = \arg \min_j \|z_e(X) - e_j\|_2$$

- ▶ **Decoder:**  $\hat{X} = \text{dec}(z_q(X))$ .

O latente efetivo é o **índice**  $k^*$  — apenas  $\log_2 K$  bits por posição.

# VQ-VAE: Posterior e Prior

**Posterior categórica determinística:**

$$Q(Z = k|X) = \begin{cases} 1 & \text{se } k = \arg \min_j \|z_e(X) - e_j\|_2 \\ 0 & \text{caso contrário} \end{cases}$$

**Prior uniforme:**

$$P(Z = k) = \frac{1}{K}, \quad k = 1, \dots, K$$

Com essas escolhas:

$$D_{\text{KL}} [Q(Z|X) \| P(Z)] = \log K$$

**Constante** em relação aos parâmetros! Logo, o termo de KL **some** da loss durante o treino do autoencoder.

# VQ-VAE: Problema do Gradiente

A operação  $\arg \min$  **não é diferenciável**.

**Solução — Straight-Through Estimator:**

- ▶ **Forward:** usa  $z_q(X)$  (vetor do codebook).
- ▶ **Backward:** copia o gradiente de  $z_q$  para  $z_e$  diretamente.

Em PyTorch:

$$z_q = z_e + \text{stop\_grad}(z_q - z_e)$$

Isso permite treinar o encoder, mas **não atualiza o codebook**: precisamos de termos adicionais na loss.

# VQ-VAE: Loss

Três termos:

$$\begin{aligned}\mathcal{L} = & \underbrace{\|X - \text{dec}(z_q(X))\|^2}_{\text{reconstrução}} \\ & + \underbrace{\|\text{sg}[z_e(X)] - e\|^2}_{\text{codebook loss}} \\ & + \beta \underbrace{\|z_e(X) - \text{sg}[e]\|^2}_{\text{commitment loss}}\end{aligned}$$

Onde  $\text{sg}[\cdot]$  é o operador *stop-gradient* e  $e$  é o vetor do codebook escolhido.

- ▶ **Reconstrução:** treina encoder e decoder.
- ▶ **Codebook loss:** move  $e_k$  em direção ao output do encoder.
- ▶ **Commitment loss:** força o encoder a se “comprometer” com algum  $e_k$ , evitando crescimento ilimitado (tipicamente  $\beta \approx 0.25$ ).

# VQ-VAE: Latente 2D para Imagens

Para imagens, o encoder produz um **mapa**  $z_e(X) \in \mathbb{R}^{h \times w \times D}$ .

Quantização **posição-a-posição**:

$$z_q(X)_{i,j} = e_{k_{i,j}^*}, \quad k_{i,j}^* = \arg \min_k \|z_e(X)_{i,j} - e_k\|_2$$

Resultado: cada imagem vira uma **grade**  $h \times w$  de índices inteiros  $\in \{1, \dots, K\}$ .

**Compressão drástica:** e.g. imagem  $256 \times 256 \times 3 \rightarrow$  grid  $32 \times 32$  de índices (com  $K = 512$  *tokens*).

# VQ-VAE: A Compressão é em *Bits*, não em Floats

**Cuidado:** a compressão **não** se mede pelo tensor contínuo

$z_q(X) \in \mathbb{R}^{h \times w \times D}$  (que pode até ter *mais* números que a imagem!), e sim pela **informação** da grade de índices.

- ▶ Cada posição guarda **um índice**  $\in \{1, \dots, K\}$ , não um vetor de  $D$  floats.
- ▶ Os vetores  $e_k \in \mathbb{R}^D$  vêm de *lookup* no codebook, recuperados deterministicamente.
- ▶ O codebook ( $K \times D$ ) é **compartilhado** por todas as imagens  $\Rightarrow$  amortizado, não conta por imagem.

Conta em bits (imagem  $256 \times 256 \times 3$ , grid  $32 \times 32$ ,  $K = 512$ ):

imagem :  $256 \cdot 256 \cdot 3 \cdot 8 = 1\,572\,864$  bits

índices :  $32 \cdot 32 \cdot \underbrace{\log_2 512}_9 = 9\,216$  bits  $\Rightarrow \approx 170 \times$  menos.

**$D$  não entra na razão** — comprimem a redução espacial e o alfabeto  $K$ .

# VQ-VAE: Como Gerar Novas Amostras?

Após treinar o autoencoder, queremos **amostrar** novos  $X$ .

**Problema:** o prior  $P(Z) = 1/K$  é uniforme — amostrar uniforme produz ruído.

**Solução em dois passos:**

1. Codifica todo o dataset em **grids de índices**  $\{k_{i,j}\}$ .
2. Treina um modelo **autorregressivo** sobre os índices (PixelCNN, Transformer):

$$P_{\psi}(k_{1,1}, k_{1,2}, \dots, k_{h,w}) = \prod_{i,j} P_{\psi}(k_{i,j} | k_{<i, <j})$$

Geração:

- ▶ Amostra um grid de índices da prior autorregressiva.
- ▶ Mapeia índices  $\rightarrow$  vetores do codebook.
- ▶ Decodifica via dec.

# VQ-VAE: O que o Transformer Recebe?

Ao usar um **transformer** como prior, o que entra são os **índices**  $k_{i,j} \in \{1, \dots, K\}$  — **não** os vetores do codebook. É *next-token prediction* sobre um vocabulário de tamanho  $K$ , igual a linguagem.

**Duas tabelas de embedding distintas** (o ponto que confunde):

- ▶ **Codebook do VQ-VAE**  $\mathcal{E} \in \mathbb{R}^{K \times D}$ : usado só no autoencoder — quantizar (encoder) e *lookup* final (decoder).
- ▶ **Embedding do transformer**  $\in \mathbb{R}^{K \times d_{\text{model}}}$ : aprendido **separadamente**, mapeia cada índice para a dimensão interna do transformer.

O **índice** é o único elo entre os dois mundos:

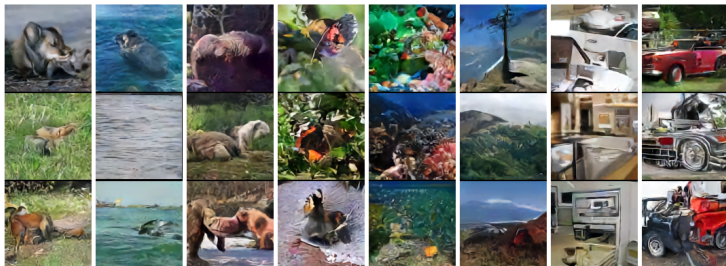
$$X \xrightarrow{\text{enc, quant.}} \{k_{i,j}\} \xrightarrow{\text{transformer}} \{k_{i,j}\} \xrightarrow{\text{lookup } e_k} \text{dec}(\cdot) \rightarrow \hat{X}$$

A saída é uma **categórica sobre  $K$  classes** (softmax) — esquema do VQGAN e do DALL-E 1.

- [2] 2021 — Patrick Esser et al. — *Taming Transformers for High-Resolution Image Synthesis*  
[6] 2018 — Aaron van den Oord et al. — *Neural Discrete Representation Learning*  
[7] 2021 — Aditya Ramesh et al. — *Zero-Shot Text-to-Image Generation*



# VQ-VAE: Amostras do Prior Autorregressivo



Topo: amostras  $128 \times 128$  do ImageNet (kit fox, urso, borboleta, coral, etc.) geradas via PixelCNN prior. Baixo: amostras do ambiente DeepMind Lab.

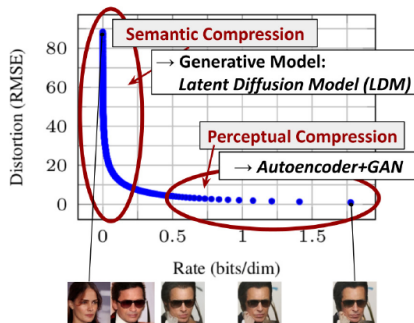
---

# Stable Diffusion

---



# Stable Diffusion: Compressão em Dois Estágios



O autoencoder perceptual comprime até o “ponto ideal” da curva (alta qualidade, baixa taxa de bits). O LDM modela semanticamente nesse espaço — um passo mais agressivo que um autoencoder puro.

# Stable Diffusion: Estratégia

## Dois estágios:

### 1. Estágio 1 — Autoencoder Perceptual:

- ▶ Encoder  $\text{enc} : X \mapsto Z$  comprime para um espaço latente **pequeno**.
- ▶ Decoder  $\text{dec} : Z \mapsto \hat{X}$  reconstrói com qualidade perceptual.
- ▶ Treinado **uma única vez** sobre um dataset grande.

### 2. Estágio 2 — Difusão Latente:

- ▶ Modelo de difusão  $g_t$  opera **dentro** do espaço latente  $Z$ .
- ▶ Condicionamento via *cross-attention*.
- ▶ Inferência: amostra  $Z_0$  via difusão  $\rightarrow$  decodifica  $\text{dec}(Z_0) = \hat{X}$ .

O espaço latente é  $\sim 48\times$  menor (e.g.  $64 \times 64 \times 4$  para imagens  $512 \times 512 \times 3$ ).



# Estágio 1: Autoencoder Perceptual — Loss

O autoencoder **não** é treinado apenas com  $\ell_2$  ou  $\ell_1$ . A loss combina vários termos:

$$\begin{aligned}\mathcal{L}_{\text{AE}}(\text{enc}, \text{dec}) = & \underbrace{\|X - \hat{X}\|_1}_{\text{reconstrução}} \\ & + \lambda_{\text{LPIPS}} \underbrace{\mathcal{L}_{\text{LPIPS}}(X, \hat{X})}_{\text{perceptual}} \\ & + \lambda_{\text{adv}} \underbrace{\mathcal{L}_{\text{adv}}(X, \hat{X})}_{\text{adversarial}} \\ & + \lambda_{\text{reg}} \underbrace{\mathcal{L}_{\text{reg}}(Z)}_{\text{regularização}}\end{aligned}$$

Cada termo tem um papel:

- ▶  $\ell_1$ : fidelidade pixelar.
- ▶ LPIPS: similaridade perceptual via features de VGG.
- ▶ Adversarial: *patch-based discriminator* (PatchGAN) para detalhes nítidos.
- ▶ Regularização: evita latente arbitrário.

# Estágio 1: LPIPS e Discriminador Patch-Based

**LPIPS** (*Learned Perceptual Image Patch Similarity*):

- ▶ Distância **euclidiana** entre features de uma rede VGG pré-treinada.
- ▶ Captura similaridade **perceptual** em vez de pixelar.
- ▶ Robusto a pequenas translações e variações de luminância.

**PatchGAN Discriminator** (do pix2pix):

- ▶ Discriminador convolucional que produz mapa  $N \times N$  de *scores reais/falsos* sobre *patches* da imagem.
- ▶ Penaliza apenas detalhes **locais** de alta frequência.
- ▶ Treino adversarial **em conjunto** com o autoencoder.

Esses dois ingredientes garantem reconstruções **nítidas**, em contraste com autoencoders puramente  $\ell_2$  que produzem reconstruções borradas.



# Estágio 1: Receita de Treino do Autoencoder

- ▶ Dataset: OpenImages, ImageNet, LAION.
- ▶ Resolução:  $256 \times 256$  ou  $512 \times 512$ .
- ▶ Otimizador: Adam,  $lr \approx 4.5 \times 10^{-6}$  por GPU.
- ▶ Discriminador é **ativado depois** de  $\sim 50\,000$  iterações de *warm-up* (autoencoder estabiliza antes do GAN entrar).
- ▶ Treino multi-GPU; o autoencoder é **congelado** antes do treino do estágio 2.

**Resultado:** encoder  $\text{enc}(\cdot)$  + decoder  $\text{dec}(\cdot)$  que comprimem com perda perceptual mínima.

## Estágio 2: Difusão no Espaço Latente

A difusão é definida sobre os **latentes**  $Z = \text{enc}(X)$ :

$$Z_0 = \text{enc}(X), \quad Z_t = \sqrt{\alpha_t} Z_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Treina-se uma UNet  $g_t(Z_t, \theta_t, c)$  para prever o ruído  $\epsilon$ :

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{X, c, \epsilon, t} \left[ \|g_t(Z_t, \theta_t, c) - \epsilon\|^2 \right]$$

Onde  $c$  é o **condicionamento** (texto, classe, máscara, etc.).

A geração:

1. Amostra  $Z_T \sim \mathcal{N}(0, I)$  no espaço latente.
2. Roda o processo reverso  $T \rightarrow 0$  na UNet.
3. Decodifica:  $\hat{X} = \text{dec}(Z_0)$ .

## Estágio 2: Arquitetura da UNet

UNet baseada na ADM (Dhariwal & Nichol) com modificações:

- ▶ **Time embeddings** sinusoidais somados aos canais (como em difusão pixel-space).
- ▶ Blocos residuais com *group norm*.
- ▶ Blocos de *self-attention* nas resoluções mais baixas.
- ▶ Blocos de *cross-attention* para condicionamento (próximo slide).
- ▶ Saída: predição de ruído  $\epsilon \in \mathbb{R}^{h \times w \times c}$  no espaço latente.

Comparado a difusão pixel-space:

- ▶ UNet **menor** (latente é menor).
- ▶ Treino  $\sim 4\times$  mais rápido.
- ▶ Inferência  $\sim 10\times$  mais rápida.



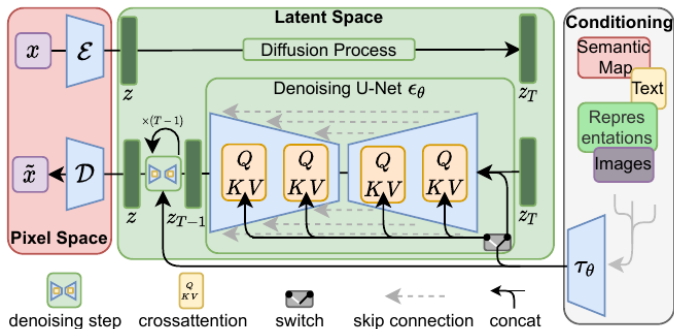
## Estágio 2: Por que Cross-Attention?

- ▶ **Flexibilidade:** o mesmo mecanismo funciona para texto, classe, máscaras. . .
- ▶ **Espacialidade:** cada posição do latente atende a tokens diferentes.
- ▶ **Composicionalidade:** “um astronauta a cavalo” → posições diferentes do latente atendem a “astronauta” e “cavalo”.
- ▶ **Escalabilidade:** cresce linearmente em  $N \cdot M$  (sequência da imagem  $\times$  sequência do texto).

Combinado com **Classifier-Free Guidance**, obtemos o *workflow* clássico de text-to-image:

- ▶ Durante treino, com prob.  $\sim 10\%$ , substitui o texto por "" (string vazia).
- ▶ Em inferência, faz duas *forward passes* (com e sem prompt) e combina com peso  $w$ .

# Stable Diffusion: Resumo da Arquitetura



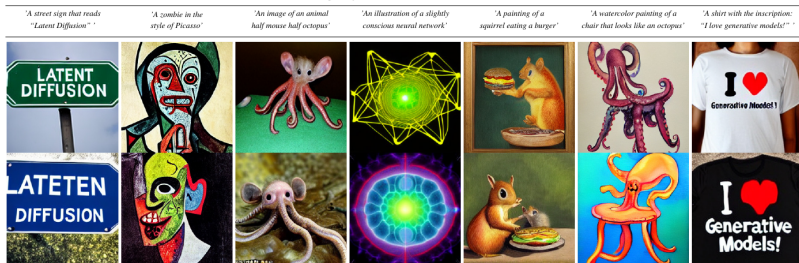
- ▶ **Estágio 1** (azul): autoencoder **congelado** após treino.
- ▶ **Estágio 2** (laranja/vermelho): difusão no latente, condicionada via *cross-attention*.
- ▶ Condicionamento  $\tau_\theta(y)$  (verde): texto via CLIP, classe via embeddings, etc.







# Aplicação: Text-to-Image



Imagens geradas a partir de *prompts* de texto (LAION,  $256 \times 256$ ). O texto é codificado por um transformer/CLIP e injetado via *cross-attention*; *prompts* arbitrários produzem composições inéditas ("a zombie in the style of Picasso", "half mouse half octopus").

# Aplicação: Inpainting (Remoção de Objetos)



Esq.: *input* mascarado (contorno vermelho). Dir.: resultado. Regenera **apenas** a área removida (**outpainting**: o caso dual).

# Aplicação: Layout-to-Image e Síntese Semântica



**Layout-to-image (COCO):** *bounding boxes* rotuladas (esq.) → cenas que respeitem a disposição espacial.



**Síntese semântica:** treinado em  $256^2$ , generaliza para  $512 \times 1024$  (paisagens).

# Aplicação: Adaptações do Ecossistema

Métodos **pós-treino** que reaproveitam o modelo base (não estão no paper original):

- ▶ **ControlNet**: ramo treinável que injeta condicionamento estrutural (bordas, poses, *scribbles*, profundidade) na UNet, **congelando** os pesos originais.
- ▶ **DreamBooth**: *fine-tuning* que ensina um **sujeito específico** (e.g. um cachorro particular) ligado a um *token* raro.
- ▶ **Textual Inversion**: aprende um **novo embedding de texto** para um conceito, sem alterar a rede.
- ▶ **LoRA**: *fine-tuning* eficiente via matrizes de baixo posto  $\Rightarrow$  adaptadores leves (poucos MB) para estilo/conceito.
- ▶ **Depth-to-image**: condiciona em um mapa de profundidade  $\rightarrow$  preserva a geometria 3D ao reescrever a aparência.

Adaptar é barato: a difusão opera no latente compacto  $Z$ .

- [11] 2023 — Lvmin Zhang et al. — *Adding Conditional Control to Text-to-Image Diffusion Models*  
[9] 2022 — Nataniel Ruiz et al. — *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*  
[3] 2022 — Rinon Gal et al. — *An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion*  
[5] 2021 — Edward J. Hu et al. — *LoRA: Low-Rank Adaptation of Large Language Models*

# Stable Diffusion: Amostras



LDMs treinados em CelebA-HQ, FFHQ, LSUN-Churches, LSUN-Beds e ImageNet condicional ( $256 \times 256$ ).

# Stable Diffusion: Super-Resolução (4 $\times$ )



*LDM*-SR: 4 $\times$  super-resolução no ImageNet-Val. Interpolação bicúbica (esq.) vs. *LDM*-SR (centro) vs. SR3 (dir.).

# Resumo da Aula

## 1. Difusão Condicionada:

- ▶ Classifier guidance:  $\tilde{g}_t = g_t - s\sqrt{1 - \alpha_t} \nabla \log P_\phi(Y|Z_t)$ .
- ▶ Classifier-free guidance:  $\tilde{g}_t = (1 + w) g_t(Z_t, Y) - w g_t(Z_t, \emptyset)$ .
- ▶ CFG é o padrão moderno.

## 2. VQ-VAE:

- ▶ Latente discreto via codebook.
- ▶ *Straight-through* para o gradiente.
- ▶ Geração: autoencoder + prior autorregressivo sobre os índices.

## 3. Stable Diffusion:

- ▶ Autoencoder perceptual (LPIPS + PatchGAN + KL/VQ-reg).
- ▶ Difusão no espaço latente.
- ▶ Condicionamento via cross-attention.

# Referências I

- [1] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. arXiv:2105.05233 [cs]. 2021. DOI: 10.48550/arXiv.2105.05233. URL: <http://arxiv.org/abs/2105.05233>.
- [2] Patrick Esser, Robin Rombach, and Björn Ommer. *Taming Transformers for High-Resolution Image Synthesis*. arXiv:2012.09841 [cs]. June 2021. DOI: 10.48550/arXiv.2012.09841. URL: <http://arxiv.org/abs/2012.09841> (visited on 08/15/2024).
- [3] Rinon Gal et al. *An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion*. arXiv:2208.01618 [cs]. 2022. DOI: 10.48550/arXiv.2208.01618. URL: <http://arxiv.org/abs/2208.01618>.
- [4] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. arXiv:2207.12598 [cs]. 2022. DOI: 10.48550/arXiv.2207.12598. URL: <http://arxiv.org/abs/2207.12598>.

## Referências II

- [5] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685 [cs]. 2021. DOI: 10.48550/arXiv.2106.09685. URL: <http://arxiv.org/abs/2106.09685>.
- [6] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. *Neural Discrete Representation Learning*. arXiv:1711.00937 [cs]. 2018. DOI: 10.48550/arXiv.1711.00937. URL: <http://arxiv.org/abs/1711.00937>.
- [7] Aditya Ramesh et al. *Zero-Shot Text-to-Image Generation*. arXiv:2102.12092 [cs]. 2021. DOI: 10.48550/arXiv.2102.12092. URL: <http://arxiv.org/abs/2102.12092>.
- [8] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. arXiv:2112.10752 [cs]. 2022. DOI: 10.48550/arXiv.2112.10752. URL: <http://arxiv.org/abs/2112.10752>.

# Referências III

- [9] Nataniel Ruiz et al. *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*. [arXiv:2208.12242 \[cs\]](#). 2022. DOI: [10.48550/arXiv.2208.12242](#). URL: <http://arxiv.org/abs/2208.12242>.
- [10] Jiaming Song, Chenlin Meng, and Stefano Ermon. *Denoising Diffusion Implicit Models*. [arXiv:2010.02502 \[cs\]](#). 2022. DOI: [10.48550/arXiv.2010.02502](#). URL: <http://arxiv.org/abs/2010.02502>.
- [11] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. *Adding Conditional Control to Text-to-Image Diffusion Models*. [arXiv:2302.05543 \[cs\]](#). 2023. DOI: [10.48550/arXiv.2302.05543](#). URL: <http://arxiv.org/abs/2302.05543>.